



指南 > 应用服务 > Map Kit（地图服务） > 路径规划 > 出行路线规划

出行路线规划

更新时间: 2025-12-26 14:19



场景介绍

从5.1.1(19)开始，支持公共交通规划功能。

提供两点之间驾车、步行、骑行和公共交通的路径规划能力。其中驾车路径规划支持添加途经点。

接口说明

以下是路径规划功能相关接口，主要由navi命名空间下的方法提供，更多接口及使用方法请参见[接口文档](#)。

接口名	描述
<code>getDrivingRoutes(params: DrivingRouteParams): Promise<RouteResult></code>	驾车路径规划。
<code>getDrivingRoutes(context: common.Context, params: DrivingRouteParams): Promise<RouteResult></code>	驾车路径规划。支持传入Context上下文。
<code>getWalkingRoutes(params: RouteParams): Promise<RouteResult></code>	步行路径规划。
<code>getWalkingRoutes(context: common.Context, params: RouteParams): Promise<RouteResult></code>	步行路径规划。支持传入Context上下文。

接口名	描述
<code>getCyclingRoutes(params: RouteParams): Promise<RouteResult></code>	骑行路径规划。
<code>getCyclingRoutes(context: common.Context,</code>	骑行路径规划。支持传

🔍 筛选目录下的文档标题



▼ Map Kit（地图服务）

<code>getTransitRoutes(context: common.Context, params: TransitRouteParams): Promise<TransitRouteResult></code>	公共交通规划。支持传入Context上下文。
<code>DrivingRouteParams</code>	驾车路径规划的参数。
<code>RouteParams</code>	步行、骑行路径规划参数。
<code>TransitRouteParams</code>	公共交通规划参数。
<code>RouteResult</code>	路径规划的结果。
<code>TransitRouteResult</code>	公共交通规划的结果。

本文导读

场景介绍

接口说明

开发步骤

驾车路径规划

步行路径规划

骑行路径规划

公共交通规划

开发步骤

导入相关模块。

```
import { navi } from '@kit.MapKit';
import { BusinessError } from '@kit.BasicServicesKit';
```

驾车路径规划

根据起终点坐标检索符合条件的驾车路径规划方案。支持以下功能：

- 支持一次请求返回多条路线，最多支持3条路线。
- 最多支持5个途经点。
- 支持未来出行规划。
- 支持根据实时路况进行合理路线规划。

► 附录

► Notification Kit（用户通知服务）

- 支持多种路线偏好选择，如时间最短、避免经过收费的公路、避开高速公路、距离优先等。

```
async testDrivingRoutes() {
  let params: navi.DrivingRouteParams = {
    // 起点的经纬度
    origins: [{
      latitude: 31.982129213545843,
      longitude: 120.27745557768591
    }],
    // 终点的经纬度
    destination: {
      latitude: 31.986129213545843,
      longitude: 120.32745557768591
    },
    // 路径的途经点
    waypoints: [{
      latitude: 31.967236140819114,
      longitude: 120.27142088866847
    }, {
      latitude: 31.972868002238872,
      longitude: 120.2943211817165
    }, {
      latitude: 31.98469327973332,
      longitude: 120.29101107384068
    }],
    language: 'zh_CN'
  };
  try {
    const result = await navi.getDrivingRoutes(params);
    console.info(`Succeeded in getting driving routes. result is ${JSON.stri
  } catch (error) {
    const err: BusinessError = error as BusinessError;
    console.error(`Failed in getting driving routes. Code is ${err.code}, me
  }
}
```

步行路径规划

根据起终点坐标检索符合条件的步行路径规划方案。支持以下功能：

- 支持直线距离150km以内的步行路径规划能力。
- 融入出行策略（时间最短、避免轮渡）。

>

```
async testWalkingRoutes() {
  let params: navi.RouteParams = {
    // 起点的经纬度
    origins: [{
      latitude: 39.992281,
      longitude: 116.31088
    }, {
      latitude: 39.996,
      longitude: 116.311
    }],
    // 终点的经纬度
    destination: {
      latitude: 39.94,
      longitude: 116.311
    },
    language: 'zh_CN'
  };
  try {
    const result = await navi.getWalkingRoutes(params);
    console.info(`Succeeded in getting walking routes. result is ${JSON.stri
  } catch (error) {
    const err: BusinessError = error as BusinessError;
    console.error(`Failed in getting walking routes. Code is ${err.code}, me
  }
}
```

骑行路径规划

根据起终点坐标检索符合条件的骑行路径规划方案。支持以下功能：

- 支持直线距离500km以内的骑行路径规划能力。
- 融入出行策略（时间最短、避免轮渡）。

```
async testCyclingRoutes() {
  let params: navi.RouteParams = {
```

```
// 起点的经纬度
origins: [{
  latitude: 31.9844102,
  longitude: 118.7662537
}],
// 终点的经纬度
destination: {
  latitude: 31.9874102,
  longitude: 118.7362537
},
language: 'zh_CN'
};
try {
  const result = await navi.getCyclingRoutes(params);
  console.info(`Succeeded in getting cycling routes. result is ${JSON.stri
} catch (error) {
  const err: BusinessError = error as BusinessError;
  console.error(`Failed in getting cycling routes. Code is ${err.code}, me
}
}
```

公共交通运输规划

根据起点终点坐标规划道路，从而返回两地之间的多种公共交通中转路线，仅支持中国大陆。

```
async testGetTransitRoutes() {
  let params: navi.TransitRouteParams = {
    // 起点经纬度
    origin: {
      latitude: 39.921619,
      longitude: 116.356587
    },
    // 终点经纬度
    destination: {
      latitude: 39.94161,
      longitude: 116.353621
    },
    // 预计出发时间
    departureTime: new Date().getTime() / 1000
  }
}
```

```
};
try {
  const result = await navi.getTransitRoutes(this.getUIContext().getHostCo
  console.info(`Succeeded in getting transit routes. result is ${JSON.stri
} catch (error) {
  const err: BusinessError = error as BusinessError;
  console.error(`Failed in getting transit routes. Code is ${err.code}, me
}
}
```

< 路径规划 批量算路 >

相关推荐

- 文档 批量算路
- 文档 业务介绍
- 文档 轨迹绑路
- 文档 如何使用内置的js引擎...
- 文档 类
- 文档 气泡
- 文档 结构体
- 文档 Pen Kit手写套件是否...
- 文档 枚举
- 视频课程 oCPC进阶功能及最新...

意见反馈

以上内容对您是否有帮助？   [意见反馈](#)

如果您有其他疑问，您可以通过开发者社区问答频道来和我们联系探讨。

[社区提问](#) [智能客服提问](#)